6. Neural Network: Multi-Class Classification

But not just any type of multi-class classification. The ones we work with nowadays consist of around 46 labels!!!

Previous Section:

5. Neural Networks: Binary Classification

Contents:

Let's review about something that *looks* familiar, but quickly stops being simple. You already know classification. Binary classification? Easy. Positive or Negative. Yes or No. Spam or Not Spam. Even small multi-class problems feel manageable. 3 classes, 4 classes, maybe 5 if you're feeling brave.

But then reality shows up. Not 3 labels. Not 5. → 10, 20, 50, 100 (in the Reuters dataset we have **46 labels**). And suddenly... everything changes.

At first glance, you might think: ***"Okay... just use the same Machine Learning models, right?"*** → And technically, you *can*.

But here's the problem. Traditional Machine Learning models were never really designed for this scale of classification. They can handle a few classes comfortably, but as the number of labels grows:

- The decision boundaries become more complex

- The relationships between classes become harder to capture
- And the model starts to struggle, not because it's broken, but because the problem itself becomes **too rich.**

Now think about what's actually happening. In a dataset like Reuters:

- Each input is text (already complex)
- Each label represents a different topic
- And some topics are **very similar to each other**

So the model isn't just separating classes anymore. It's trying to understand **subtle differences in meaning**. And that's not something simple models are good at.

This is exactly where Neural Networks come in. Because instead of forcing a model to draw rigid boundaries. Neural Networks learn **representations**.

They don't just ask: ***"Which side of the line does this belong to?"***

They ask something deeper: ***"What is this data... and how does it relate to everything else?"***

And when you combine that with something like **Softmax**, you no longer get a single yes/no answer, you get a **distribution of probabilities across all 46 classes**.

Not: ***"This is class A."*** → But: ***"This is most likely class A... but it could also be B or C."***

That subtle shift? That's what makes large-scale multi-class classification possible.

So when we talk about Neural Networks for multi-class problems, we're not just scaling up classification. We're changing the way the model **thinks about the problem itself**. From simple separation to understanding complexity.

1. The Reuters Dataset

Let's step into the dataset we'll be working with. This one is a bit different. Not because it's complicated on the surface— but because of what it quietly represents underneath.

You are given a collection of news articles. Short ones. Clean. Structured just enough to work with. Around $11,228$ **samples**, originally collected and introduced back in **1986**.

At first glance, it feels simple:

- Each sample is just a short newswire
- Each one talks about a specific topic

But here's where things start to shift. There aren't just a few topics. There are 46.

Now imagine yourself in that situation. You read a piece of news. Maybe it talks about economics, maybe politics, maybe trade. And you are asked one question: ***"Which category does this belong to?"*** → Not multiple answers. Not "pick all that apply". **Exactly one.**

And that detail matters more than it seems. Even though the dataset has 46 labels, each article belongs to **one—and only one—class**. So despite the scale, this is **not** a multi-label problem. It is a **multi-class classification problem**.

From a distance, it might look like just another dataset. But if you look closer. You'll realize what it's really testing:

- Can a model understand **language**?
- Can it distinguish between **subtle differences in topics**?
- Can it confidently choose **one class out of 46**, even when some of them look very similar?

And that's exactly why this dataset exists in this chapter. Because this is the moment where classification stops being trivial, and starts becoming **understanding**.

2. Train a Neural Network for Multi-Class Classification Problem

Now we move from understanding the problem to actually **building something that can solve it**. And as always, we don't jump straight into the model. We observe first. We prepare. We structure the data so that the model can truly *learn from it*.

Step 1 — Load the Dataset

We begin by loading the Reuters dataset. Just like before, we limit ourselves to the **10,000 most frequent words**. Not because the rest are useless. But because in language, a small subset of words carries most of the meaning.

```
from tensorflow.keras.datasets import reuters
import numpy as np

# Load the Reuters dataset (keep the 10,000 frequent words
just like the IMDB)
(x_train, y_train), (x_test, y_test) = reuters.load_data(nu
m_words=10000)

# Display the sizes and number of unique labels
print("Train Size:", x_train.shape)
print("Test Size:", x_test.shape)
print("Unique labels:", len(np.unique(y_train)))

# Display the first sample
print("First samples:", len(x_train[0]), "; Label:", y_trai
n[0])
```

```
Train Size: (8982,)
Test Size: (2246,)
Unique labels: 46
First samples: 87 ; Label: 3
```

At this stage, what we see is not “text”. It is sequences of numbers. Because before we can understand meaning, we must first accept how machines see the world.

Step 2 — Vectorization & One-Hot Encoding

Now we transform the data. Not to simplify it, but to make it **learnable**. We convert each sequence into a **binary vector of size 10,000**. Each position answers a simple question: ***“Does this word exist in this article?”***

```
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.0

    return results
```

Then the labels. Even though we have **46 classes**, we still encode them using **one-hot encoding**. Because the model must not see labels as numbers, but as **distinct possibilities**.

```
def to_one_hot(labels, dimension=46):
    results = np.zeros((len(labels), dimension))
    for i, label in enumerate(labels):
        results[i, label] = 1.0
    return results
```

Apply everything:

```
x_train = vectorize_sequences(x_train, 10000)
x_test = vectorize_sequences(x_test, 10000)

y_train = to_one_hot(y_train, 46)
y_test = to_one_hot(y_test, 46)

print('Shape x_train:', x_train.shape)
print('Shape x_test :', x_test.shape)
print('Shape y_train:', y_train.shape)
print('Shape y_test :', y_test.shape)
```

```
Shape x_train: (8982, 10000)
Shape x_test : (2246, 10000)
Shape y_train: (8982, 46)
Shape y_test : (2246, 46)
```

If the label shape is `(samples, 46)`, then we know we did it right.

Step 3 — Validation Set

Before training, we pause. Because learning without validation, is like observing without reflection. We set aside part of the training data:

```
# Validation set for the training process
x_val_reuters = x_train[:1000]
partial_x_train_reuters = x_train[1000:]
y_val_reuters = y_train[:1000]
partial_y_train_reuters = y_train[1000:]

print("Shape x_val_reuters:", x_val_reuters.shape)
print("Shape partial_x_train_reuters:", partial_x_train_reu
ters.shape)
print("Shape y_val_reuters:", y_val_reuters.shape)
print("Shape partial_y_train_reuters:", partial_y_train_reu
ters.shape)
```

```
Shape x_val_reuters: (1000, 10000)
Shape partial_x_train_reuters: (7982, 10000)
Shape y_val_reuters: (1000, 46)
Shape partial_y_train_reuters: (7982, 46)
```

This is where we will measure... Not how well the model memorizes... but how well it **understands**.

Step 4 — Build the Neural Network

We construct the model. Simple structure. But powerful enough to learn complex patterns.

```
from tensorflow import keras
from tensorflow.keras import layers

# Training the model
hidden_layer1 = layers.Dense(64, activation='relu')
hidden_layer2 = layers.Dense(64, activation='relu')

# Output Layer needs 46 neurons → 46 probabilities
output_layer = layers.Dense(46, activation='softmax')

reuters_model = keras.Sequential([hidden_layer1, hidden_lay
er2, output_layer])
```

Notice the final layer. Not sigmoid. Not linear. **Softmax.** Because we are not asking: ***"Is this class correct?"*** → We are asking: ***"How likely is each of the 46 classes?"***

Step 5 — Compile & Train

We now define how the model learns.

```
# Compile
reuters_model.compile(optimizer='rmsprop',
                      loss='categorical_crossentropy', metr
ics=['accuracy'])
```

And then we let it learn.

```
# Fit the model
history_reuters = reuters_model.fit(partial_x_train_reuter
s, partial_y_train_reuters,
                                    epochs=20, batch_size=25
6,
                                     validation_data=(x_val_
reuters, y_val_reuters),
                                    verbose=1)
# Save to history for visualization
history = history_reuters.history
```

This loop repeats over and over:
$$\text{Prediction} \rightarrow \text{Error} \rightarrow \text{Adjustment} \rightarrow \text{Improvement}$$

Until the model begins to **see patterns we cannot explicitly describe**.

Step 6 — Observe the Learning Process

We don't stop at training. We observe. Because the model's behavior tells us more than the final accuracy ever could.

```
import matplotlib.pyplot as plt

history_dict = history_reuters.history
loss = history_dict['loss']
```

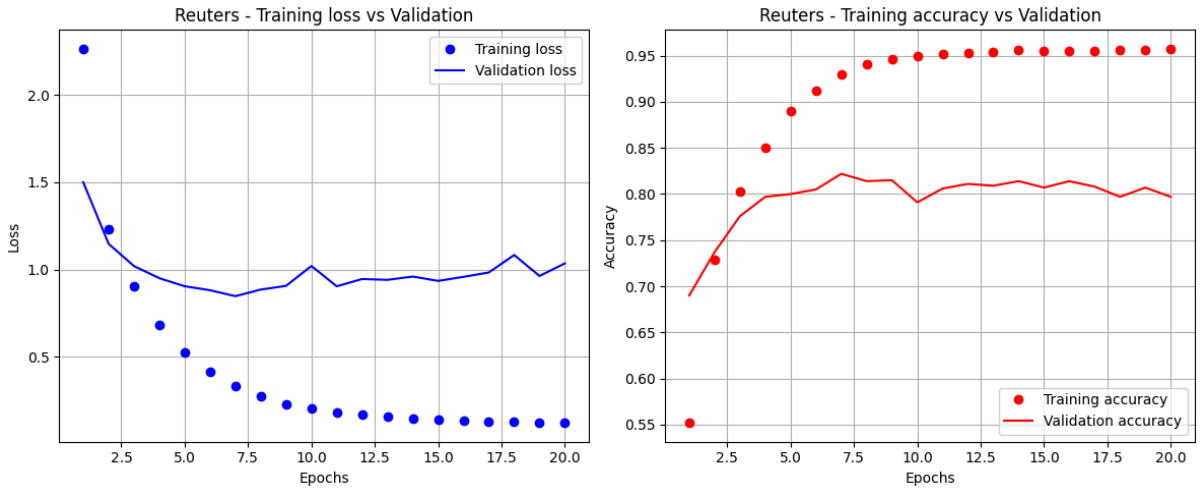
```
val_loss = history_dict['val_loss']
acc = history_dict['accuracy']
val_acc = history_dict['val_accuracy']
epochs = range(1, len(loss) + 1)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# Loss
ax[0].plot(epochs, loss, 'bo', label='Training loss')
ax[0].plot(epochs, val_loss, 'b', label='Validation loss')
ax[0].set_title('Reuters - Training loss vs Validation')
ax[0].set_xlabel('Epochs')
ax[0].set_ylabel('Loss')
ax[0].legend()
ax[0].grid(True)

# Accuracy
ax[1].plot(epochs, acc, 'ro', label='Training accuracy')
ax[1].plot(epochs, val_acc, 'r', label='Validation accurac
y')
ax[1].set_title('Reuters - Training accuracy vs Validatio
n')
ax[1].set_xlabel('Epochs')
ax[1].set_ylabel('Accuracy')
ax[1].legend()
ax[1].grid(True)

plt.tight_layout()
plt.show()
```



At some point, you will notice something subtle. Validation loss stops decreasing. Validation accuracy stops improving. And beyond that point, the model is no longer learning. It is **memorizing**.

So we do what we always do. We observe. We adjust. We retrain. Using the epoch where the model understood the data best—not where it tried the hardest. Because in the end, we are not training a model to be perfect. We are training it to **understand just enough to generalize beyond what it has seen**.

Step 7 — Inspect the Predictions

Before we move on, we do something simple. But very important. We **look at what the model actually believes**. Not just the final answer, but the **entire probability distribution**.

We take the first sample from the test set and ask the model: ***"What do you think this is?"***

```python
# Predict probabilities for the first test sample
predictions = reuters_model.predict(x_test)

first_prediction = predictions[0]

print("Predicted probabilities:", first_prediction)
print("Sum of probabilities:", np.sum(first_prediction))
print("Predicted label:", np.argmax(first_prediction))
print("Actual label   :", np.argmax(y_test[0]))
```

```
Predicted probabilities: [1.51527945e-06 1.23616374e-07 1.94741062e-07 9.42407668e-01
5.69067784e-02 1.70681058e-09 4.99595316e-08 1.04956337e-06
2.04691445e-04 2.06202238e-08 1.03318894e-06 1.21770885e-04
3.81964540e-08 4.07142943e-05 7.92734944e-09 5.47998180e-09
7.65922086e-05 6.57054899e-09 1.87578621e-07 1.97901295e-06
1.15870360e-04 5.84597547e-05 3.87419229e-11 3.01934278e-07
5.49568213e-09 5.31178799e-08 1.09388756e-08 2.90271829e-09
1.85023585e-09 4.17350338e-06 2.38056600e-06 2.91111007e-07
2.94626943e-06 9.74592962e-10 3.10485063e-07 5.70166605e-07
1.96374804e-05 1.61470326e-09 7.58828911e-09 2.82227938e-05
7.65435271e-10 2.16072340e-06 2.38623774e-11 3.76206089e-09
4.43833026e-10 1.24018837e-10]
Sum of probabilities: 0.9999999
Predicted label: 3
Actual label   : 3
```

Now observe carefully. The output is **not a single number.** It is a vector of **46 probabilities.** And if everything is correct, the sum of all probabilities $= 1.0$ **(or really close to it)**

This is the model's way of thinking: Not certainty. But **confidence across possibilities**.

It might say: 70% → Class A; 20% → Class B; 10% → Others. And from that... we take the highest value as the final prediction. But the real insight is not the answer itself... it's the **distribution behind it**

Summary

At this point, you might feel something interesting. We went through an entirely new dataset. We handled **46 labels**. And yet, nothing fundamentally new was

introduced.

The same ideas still apply:

> ***Vectorization → One-hot encoding → Neural Networks (Training → Validation → Observation)***

So what actually changed? Not the method. **The scale. The complexity. The need for better representation.**

You could say: The Reuters dataset is new → The 46 labels are new

But the core idea? Still the same. And that's the point. When your foundation is strong, even a problem with **46 classes** becomes just another step forward.

References:

https://developers.google.com/machine-learning/crash-course/neural-networks/multi-class

https://www.kaggle.com/code/asamad06/multiclass-classification-using-neural-network

Next Section:

- 7. Neural Networks: Regression